Governing What AI Agents Can Do

SecureMCP — A Business Overview

Written for product, sales, and marketing. No engineering background assumed.

---

1. In One Page

AI assistants used to answer questions. Now they take actions — query a customer database, issue a refund, send an email, update a record, call an internal service. The industry standard that makes this possible is the Model Context Protocol (MCP). It has been adopted quickly because it works.

What MCP does not include is any notion of permission. It defines how an AI agent discovers what it can do and how it does it. It says nothing about whether the agent *should* — who authorised it, under what limits, on whose behalf, subject to which regulation, and what record survives afterward.

SecureMCP is the layer that adds those answers. The framing that fits on a slide:

> **MCP gives an AI agent capability. SecureMCP governs that capability.**

In practice, SecureMCP sits between the agent and the systems it can reach, and does four things:

	The question it answers	The business value
Control	Is this action allowed, right now, for this requester?	Actions outside policy do not happen
Proof	What happened, and can it be proven later?	A tamper-evident record an outside auditor can verify
Trust	Should this tool be available at all?	Vetted, signed tools instead of unverified ones
Response	Something looks wrong — now what?	Automatic slowdown, confirmation, or shutdown

It is built as fourteen independent capabilities. Each one is switched on separately, so a customer can start with a single capability and expand, rather than committing to an all-or-nothing platform decision. That matters commercially: the first deployment can be small.

---

2. The Problem: Agents That Act

Three shifts happened at roughly the same time, and together they created the gap SecureMCP fills.

Agents got access. An AI assistant with no connection to internal systems is limited but safe. An AI assistant connected to a CRM, a billing platform, and a document store is useful — and is now a participant in business processes that previously required a person with a login.

Access became standardised. MCP made connecting an agent to a system routine work rather than a custom integration project. That is why adoption moved fast, and it is also why the number of connected systems per agent is growing rather than shrinking.

Nothing standardised the guardrails. The protocol leaves permission, limits, audit, and oversight entirely to whoever builds the server. In practice that means each team invents its own approach, or defers the question. The result is a familiar pattern: capability arrives first, governance arrives after the first incident.

The specific risks this creates are ones any risk committee will recognise:

- An agent takes an action nobody authorised, and there is no record of who or what triggered it
- An agent handles regulated data — health records, payment details, personal data — with no evidence trail that would satisfy an auditor
- A permission granted for one narrow purpose is reused far more broadly
- An agent's behaviour changes — many more calls, unusual targets, higher error rates — and nobody notices until the damage is done
- A tool is installed from outside the organisation with no verification of who wrote it or what it claims to do

None of these are exotic. They are the ordinary consequences of giving software the ability to act without giving it the constraints a person in the same role would have.

---

3. What SecureMCP Is

The most useful comparison is a building with a security desk.

An MCP server is a building full of rooms — each room a system the agent can use. Before SecureMCP, the agent walks in and opens any door it can find. After SecureMCP, there is a desk at the entrance, a badge with specific room access, a visitor log that cannot be quietly edited, and a guard who notices when someone who normally visits two rooms suddenly tries forty.

Three properties are worth understanding, because they come up in nearly every customer conversation.

It does not replace what is already there. SecureMCP attaches to an existing MCP server rather than replacing it. Nothing has to be rebuilt, and the server keeps working the way it did. This is the difference between a governance project that takes two weeks and one that takes two quarters.

Every capability is optional and independent. There is no "security mode" that either is or is not on. Fourteen capabilities are switched on individually. A customer worried about audit turns on the record-keeping. A customer worried about regulated data turns on the rule engine. Neither is forced to take the other.

When it cannot decide, it blocks. If the rule engine cannot reach a clear answer — a rule component failed, the request cannot be described properly — the default outcome is to deny the action. This is the correct default for a governance product, and it is worth stating explicitly to buyers, because the alternative

failure mode is the one that produces incidents: a system that quietly allows everything the moment something breaks.

---

4. The Five Checkpoints

Every request an agent makes passes through five checkpoints in a fixed order before it reaches the system it is trying to use. The order is not arbitrary — each checkpoint is placed where it can do its job most cheaply and most reliably.

```
Agent request
     │
     ▼
1. Is this allowed?          — the rule engine
     │
     ▼
2. Is it within agreed terms? — the agreement check
     │
     ▼
3. Write it down             — the record
     │
     ▼
4. Does this look normal?    — the behaviour watch
     │
     ▼
5. Did the owner consent?    — the permission check
     │
     ▼
The actual system
```

1. Is this allowed? The rule engine evaluates the request against the organisation's policies. This runs first because a clear "no" should cost as little as possible.

There is a detail here worth carrying into sales conversations. When an agent asks "what can I do?", SecureMCP filters the answer to only what that requester is actually permitted to use. Tools the agent cannot use do not appear in the list at all. Capability the requester is not entitled to is not merely blocked — it is invisible. Buyers consistently find this more reassuring than a block, because it removes the temptation to probe.

2. Is it within agreed terms? Where an agent has negotiated a working agreement — what it may do, which resources it may touch, for how long — that agreement is checked. Both sides can be required to sign it cryptographically, which means neither can later claim they agreed to something different.

3. Write it down. The action is recorded. Successes and failures both, because a blocked attempt is often the more interesting entry. This is covered in section 6.

4. Does this look normal? SecureMCP builds a picture of each requester's normal behaviour — how often it calls, how fast, how often it errors — and measures each new request against that picture. The

mechanism is the same idea as credit card fraud detection: the individual purchase is not suspicious, the pattern is.

Responses escalate rather than jumping straight to a block: continue, ask a human to confirm, slow the requester down, or stop it entirely.

5. Did the owner consent? The last check before the action happens is whether the owner of the data or resource actually permitted this requester to do this thing.

---

5. The Fourteen Capabilities

Each of these is enabled independently. The right-hand column is the version to use with a non-technical buyer.

Capability	What it does, in business terms
Rule engine	Decides whether each action is allowed, and records which policy or regulation clause produced the decision
Agreements	Negotiated, signed terms of engagement between the organisation and an agent
Record	A tamper-evident log of everything that happened, verifiable by an outside party
Behaviour watch	Learns normal behaviour per requester and reacts when it changes shape
Permissions	Who has consented to what, including permissions passed from one party to another
Alerts	Emits security events so existing monitoring tools can consume them
Certification	Turns a tool's declared behaviour into a signed, verifiable claim
Trust scoring	A single trust score per tool, combining certification, track record, and age
Tool catalogue	The inventory of available tools and their standing
Federation	Lets separate organisations share trust information without giving up their own authority
Revocation list	The recall mechanism — pull a compromised tool out of service immediately
Compliance mapping	Connects controls to named regulatory frameworks for reporting
Sandbox	A declared rulebook for how a tool should behave, plus visibility when it does not (see section 10)
Gateway and dashboard	The read-only view of what the governance layer is doing

Only the first five sit on the path of every request. The rest run at publish time, at query time, or on demand — which is the answer to the performance question when it comes up.

---

6. Proof: The Part Auditors Care About

The record-keeping deserves its own section, because it is the capability that most often turns a technical evaluation into a signed contract.

Most systems keep logs. Logs have a well-known weakness: whoever controls the system controls the logs. An auditor is being asked to trust the accuracy of a record produced by the party being audited.

SecureMCP's record is built so that trust is not required.

Every entry is linked to the one before it. Think of a numbered ledger where each page carries a fingerprint of the previous page. Changing an old entry breaks every fingerprint after it. Entries cannot be quietly edited, and they cannot be quietly removed.

Any single entry can be proven on its own. A specific action can be proven to belong to the record without handing over the rest of it. This matters in practice: a dispute about one transaction does not require disclosing every other transaction to resolve it.

The proof works away from the system. SecureMCP can export a self-contained evidence package for any entry. An auditor, a regulator, or the other side of a dispute can verify it independently, on their own machine, with no access to the running system. Integrity is something they confirm, not something the vendor asserts.

Two related points for accuracy. The record captures what was asked and what the outcome was, not a copy of the data that came back — deliberately, so that sensitive results are not duplicated into a permanent store. And each record entry is anchored to a random starting value unique to that record, so a plausible-looking fake record cannot be manufactured and passed off as the real one.

How to use this in a conversation. The question "how would you prove to a regulator what your AI agent did last March?" is a strong one, because most organisations cannot answer it. This is the answer.

---

7. Permissions That Cannot Grow

Permission handling contains one design decision worth understanding, because it addresses a risk that is easy to describe and hard to solve.

Permissions in SecureMCP are specific: this requester may do this particular thing with this particular resource, optionally only under certain conditions, and optionally only until a certain date.

Permissions can also be passed along — an agent given access can delegate some of it to another agent, which is necessary in any real multi-agent system. The rule governing that is strict: **a passed-along permission can only ever be narrower than the one it came from.** Nothing can grant more than it was given. Authority shrinks as it spreads; it never grows.

Withdrawal follows the same logic. Revoking a permission automatically revokes everything derived from it. There is no orphaned access still working because it was handed off before the withdrawal.

And every decision explains itself. When access is granted through a chain of three delegations, SecureMCP can show that chain. Denials are explained the same way — which missing permission caused it, not just that something was refused.

Both properties address the risk auditors call privilege creep: access that accumulates quietly until nobody can say why a given system can reach a given dataset.

---

8. Trust: Deciding Which Tools Belong

The tools an agent can use are software, often written by someone outside the organisation. Treating them all as equally trustworthy is the same mistake as installing any application a user asks for.

SecureMCP handles this with a chain that will look familiar to anyone who has thought about app store review or food safety inspection.

A tool declares what it does. Which resources it needs, what behaviour to expect.

That declaration is certified and signed. The result is a cryptographic claim that cannot be forged or altered after the fact. Certification comes in levels, from independently audited down to self-declared, and the level is visible.

Unsigned claims count for nothing. By default, a certification without a valid signature cannot improve a tool's standing at all. An unverifiable claim has no commercial value inside the system, which is exactly the property that makes certification meaningful.

Each tool carries a trust score. Three inputs, weighted deliberately:

Input	Weight	Reasoning
Certification	50%	A verified signature is stronger evidence than opinion
Track record	35%	Accumulated behaviour, starting from neutral rather than trusted
Age	15%	Deliberately capped — longevity is weak evidence, and the design refuses to let it become strong evidence

New tools start neutral: neither trusted nor suspected. Scores update immediately when new evidence arrives, and a significant drop raises a high-priority alert.

Compromised tools can be recalled. A revocation list pulls a tool out of service immediately. Where organisations have chosen to federate, a recall can propagate to peers — but each organisation's own list remains authoritative. A peer can inform a recall decision; it cannot make one on another organisation's behalf. That distinction is what makes federation acceptable to security teams who would otherwise refuse to participate.

---

9. Compliance and Regulated Industries

Six frameworks are modelled directly: **GDPR, HIPAA, SOC 2, ISO 27001, PCI-DSS, and NIST 800-53.**

The mechanism that makes this more than a checklist: when the rule engine blocks an action, the decision can carry a citation to the specific clause that caused it — the regulation, the article, a link to the primary

source, and *which version of the text* was in force. Years later, an audit can reconstruct not only that a rule applied but which wording of it did.

The practical difference is in how an audit goes. "Show me how you prevent your AI agent from touching patient data without authorisation" is normally answered with a document describing intent, followed by weeks of evidence gathering. Here it is answered with a report generated from the system's own records.

An important accuracy point for anyone writing copy. SecureMCP helps an organisation implement and evidence controls that map to these frameworks. It does not make anyone compliant, and it is not a certification. Compliance is an organisational outcome involving process, people, and an auditor. Overstating this is the single easiest way to create a problem for the account team later — see section 12.

Where the fit is strongest:

Sector	The pressure	The relevant capabilities
Healthcare	HIPAA; patient data touched by agents	Rule engine with clause citations, record, permissions
Financial services	Audit requirements, transaction traceability	Record with exportable proof, behaviour watch, agreements
Regulated enterprise	Data residency, access review, privilege audits	Permissions that cannot expand, cascading withdrawal, compliance reports
Platforms and marketplaces	Third-party tools of unknown provenance	Certification, trust scoring, revocation, federation
Government and defence	NIST control mapping, verifiable audit	Full stack, with an OS-level sandbox alongside (section 10)

10. What SecureMCP Does Not Do

This section exists because governance products fail commercially when their claims exceed their behaviour. A buyer who discovers an overstated claim during evaluation stops trusting everything else on the slide. Stated plainly and early, these same points build credibility — they demonstrate that the boundaries were mapped deliberately.

The sandbox is a rulebook, not a locked room. This is the most important one. A tool declares how it will behave, and SecureMCP checks it against that declaration and records divergence. But the tool has to participate. A tool deliberately built to ignore the rules is not stopped by this layer.

The honest positioning: SecureMCP's sandbox gives visibility and accountability for tools that cooperate. For genuine containment, run tools inside a real operating-system sandbox and use SecureMCP as the meaning-aware layer on top. That combination is stronger than either alone, and it is a better sales story than an overstated claim, because it is one the customer's own security team will agree with.

The agreement check is permissive if it fails. If agreement evaluation itself errors, terms are accepted rather than rejected — the opposite of the rule engine's behaviour. Because the rule engine runs first and does block on failure, the categorical protections still hold. But agreements should not be sold as the primary control.

There is a development-only setting that must never ship. A configuration option allows unsigned certifications to report as valid. It exists for local development and testing. In production it produces claims that cannot be checked. It defaults to off, and it should stay off.

Local desktop mode can be configured to skip enforcement. For local use, where the parent application is trusted, enforcement can be bypassed. It is off by default and warns loudly when combined with active rules.

Identity comes from elsewhere. SecureMCP tracks a short identifier to correlate activity across its records. That is a correlation key, not proof of identity. Authenticating who is really making a request is the authentication layer's job, and SecureMCP relies on it rather than replacing it.

Some numbers are estimates. Usage counts are approximations, good for spotting unusual volume, not suitable for billing.

Trust scores need persistent storage to survive a restart. With storage configured they persist; without it they are rebuilt.

A known gap in one advanced configuration. When federation and shared permissions are used together, the current start-up sequence does not connect the two as intended. It is documented, understood, and has a straightforward fix. Worth knowing so it is not discovered by a prospect first.

---

11. Adoption: Start Small

Because every capability is independent, adoption does not require a platform decision. This is the most useful thing to know when a deal stalls on scope.

Stage 1 — See what is happening. Turn on the record. Nothing is blocked, nothing breaks, and the customer gets a verifiable log of every action their agents take. This is the easiest possible yes: no behaviour change, immediate audit value. It also tends to be persuasive on its own, because the first look at what agents are actually doing is usually surprising.

Stage 2 — Add the rules. With visibility established, the customer can see which actions matter and write policy against reality rather than speculation. Rules can be tested in simulation against real recorded scenarios before going live — which removes the "what if we block something important?" objection.

Stage 3 — Add permissions and behaviour watch. Owner consent for sensitive resources, plus automatic response to behaviour that changes shape.

Stage 4 — Add trust and certification. Relevant once the number of tools grows, or once third-party tools are in play.

Each stage delivers value on its own. None requires the next.

---

12. How to Describe It Accurately

Governance products are unusually easy to oversell, because the language of security invites absolutes. These pairings keep claims defensible.

Do not say	Say instead
"Sandboxes untrusted tools"	"Provides declared behaviour rules and visibility for cooperating tools; pair with OS-level isolation for containment"
"Prevents malicious tools"	"Detects and records divergence from declared behaviour, and can revoke a tool immediately"
"Makes you HIPAA compliant"	"Helps implement and evidence controls that map to HIPAA requirements"
"Complete AI security"	"Governs what agents are allowed to do, and proves what they did"
"Immutable"	"Tamper-evident — alteration is detectable and independently verifiable"
"Authenticates agents"	"Works with your authentication layer; enforces what the authenticated party may do"
"Blocks all unauthorised access"	"Unauthorised capability is not exposed and not permitted, with rules failing closed"

Three claims that are strong and fully supportable:

1. **Independent verification.** An outside party can verify the record without access to the running system. Few products in this space can say this.
2. **Permissions cannot expand.** Delegated authority is provably narrower than its source, and withdrawal cascades automatically.
3. **Decisions cite their source.** A block can name the exact regulation clause and text version behind it.

And one differentiator that is easy to miss: **the boundaries are documented in the product itself.** The sandbox states that it is not a containment boundary. The development-only setting says so. That is a credibility asset with security teams, who spend most of their evaluation time trying to find out what a vendor left out.

13. Glossary

Term	Plain meaning
MCP	Model Context Protocol — the industry standard for connecting AI agents to tools and data
Agent	An AI system that takes actions, not just answers questions
Tool	A single capability exposed to an agent — look up a customer, send an email, run a query
Policy	A rule about what is allowed, under what conditions
Fail closed	When the system cannot decide, it denies. The safe default
Tamper-evident	Alteration cannot be prevented, but it cannot be hidden either
Attestation	A signed, verifiable claim about what a tool does
Revocation	Pulling a tool out of service — a recall notice
Delegation	Passing part of a permission to another party
Drift	Behaviour moving away from its established normal pattern
Federation	Separate organisations sharing trust information while each keeps its own authority
Provenance	The verifiable record of what happened

14. Questions That Come Up

Does this slow things down? Only five of the fourteen capabilities run on each request, and the first thing that happens is the cheapest possible rejection. Customers who care about this should be encouraged to measure it in their own environment with their own rules — the honest answer depends on how much policy they write.

Do we have to rebuild our MCP server? No. SecureMCP attaches to an existing server. That is a core design commitment, not a convenience.

What if the governance layer itself fails? The rule engine denies rather than allowing. Availability is affected before enforcement is. For a governance product this is the correct trade, and it is worth saying out loud, because it is the opposite of how most software fails.

Can we use just one piece? Yes, and it is the recommended way to start. See section 11.

How is this different from an API gateway or existing access controls? Those govern network calls and user sessions. SecureMCP governs agent actions with awareness of what the action means — that this particular tool touches health records, in production, at high risk. It also adds things a gateway does not: verifiable proof, behaviour monitoring per agent, tool certification, and consent that cannot expand as it is delegated.

Does it work with our existing monitoring? Yes. Security events are emitted for existing tooling to consume, and there is a read-only API and dashboard.

Where does the data live? In the customer's own storage. Options range from in-memory for development to SQLite and PostgreSQL for production. Nothing is sent anywhere by default.

Is there anything we should not do with it? Three things. Do not rely on the sandbox for containment — pair it with real isolation. Do not enable the development-only signing bypass in production. Do not sell agreements as the primary control, since the rule engine is the layer that fails closed.

---

15. The Short Version

MCP gave AI agents the ability to act. Nothing in the protocol governs whether they should.

SecureMCP adds the governance: rules that decide what is allowed and fail closed when uncertain, a record that an outside party can verify without trusting the system that produced it, permissions that shrink rather than grow as they are shared, trust that must be earned through verifiable signatures, and automatic response when an agent's behaviour changes shape.

It attaches to what already exists, it turns on one capability at a time, and it documents exactly where its guarantees stop — which is what makes the guarantees it does make worth relying on.